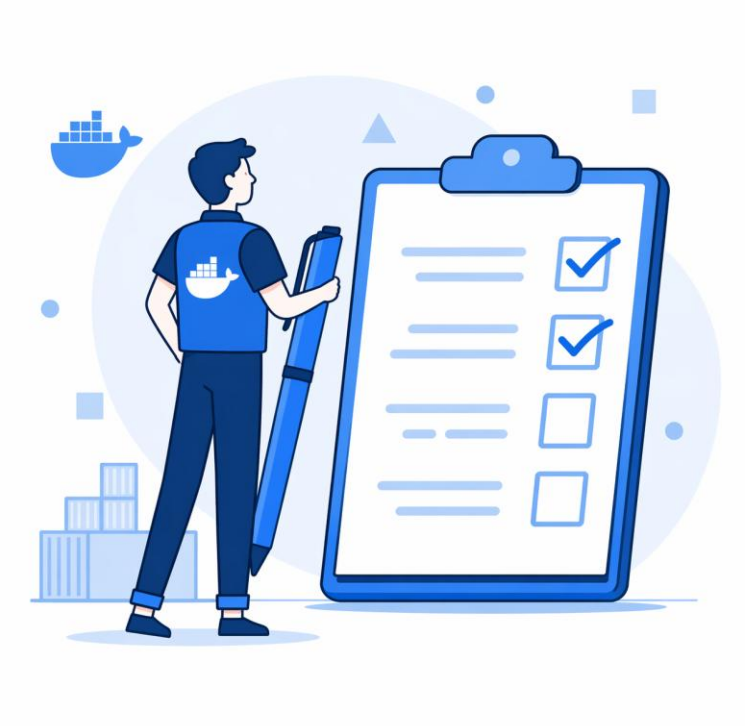
Docker: conceptos y aplicaciones en entornos de desarrollo

Ernaes Antonio Trujillo Vizuet

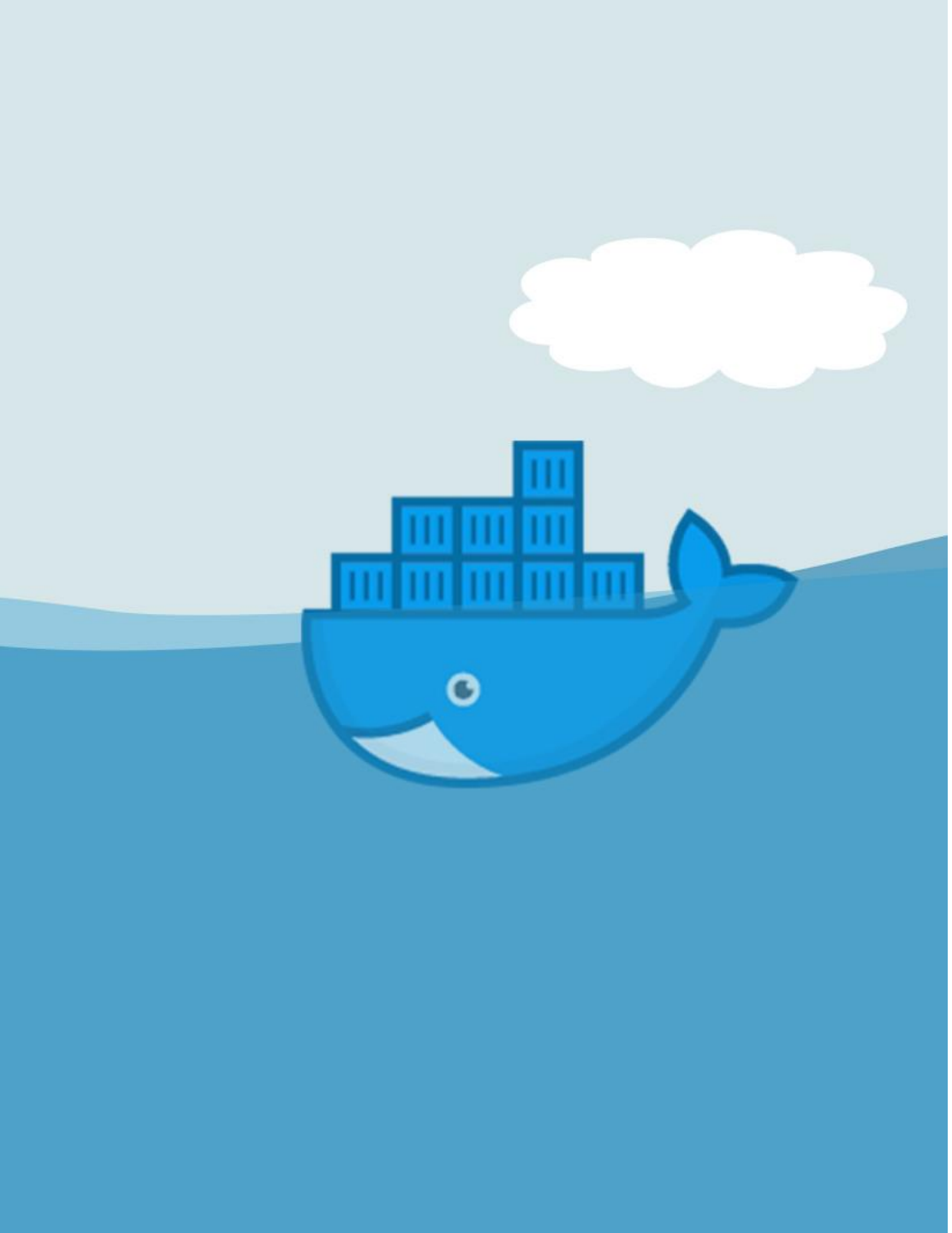
Objetivo

Aprender a contenerizar aplicaciones y desplegarlas con sus dependencias usando Docker

Tópicos del taller



1. Introducción a docker
2. Conceptos básicos
3. Ejecutar un contenedor
4. Gestión de contenedores
5. Imágenes de contenedor
6. Dockerfiles y build
7. Entrypoints, argumentos y variables de entorno
8. Volúmenes, persistencia y montaje
9. Redes Docker
10. Tu primera App



Introducción a Docker

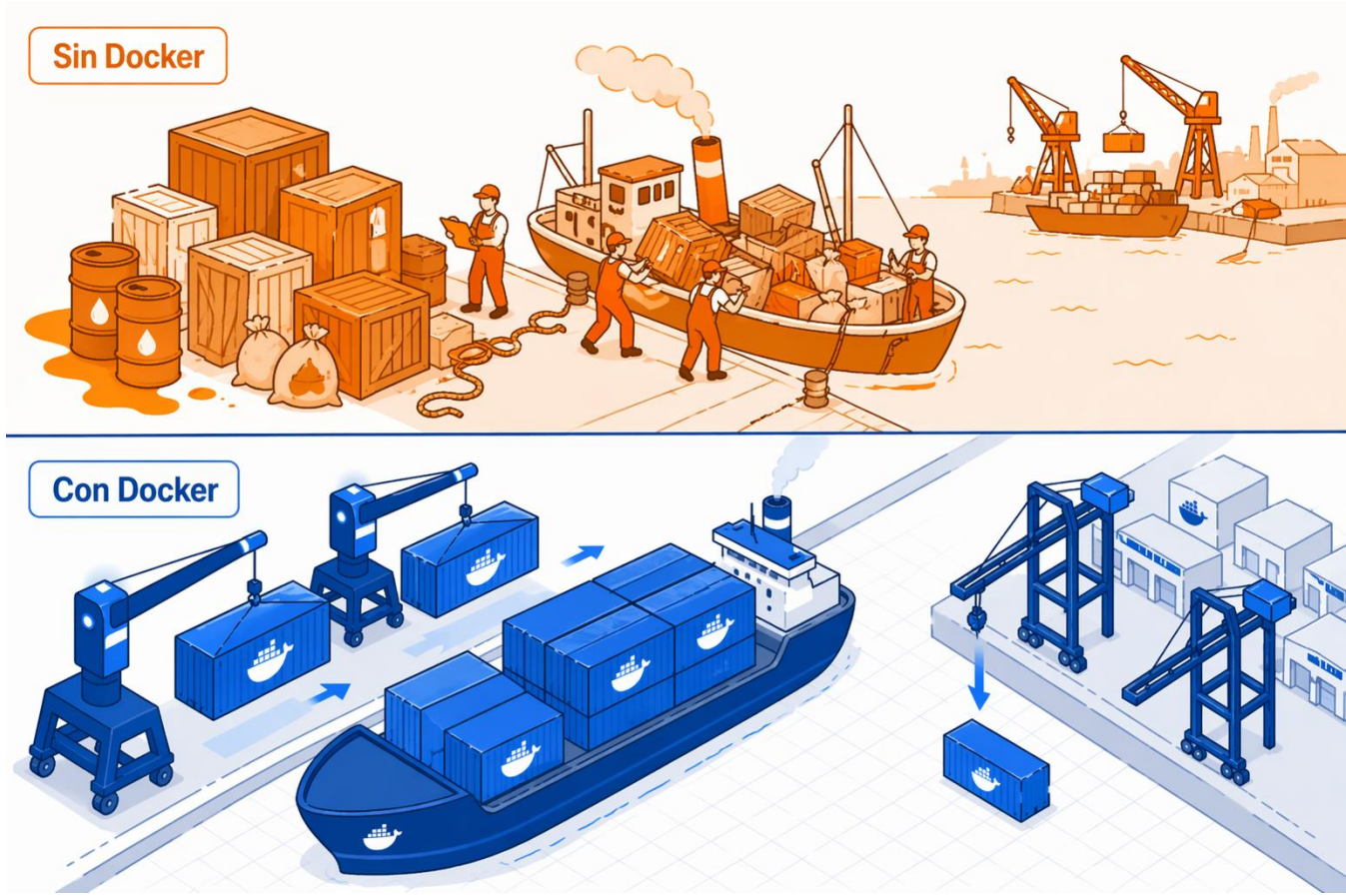
¿Qué es un contenedor?

Un contenedor es una unidad de software que encapsula una aplicación y todas sus dependencias en un entorno aislado.

Proporciona una forma de empaquetar, distribuir y ejecutar aplicaciones de manera consistente en diferentes entornos, ya sea en un entorno de desarrollo, pruebas o producción.

Antes de Docker

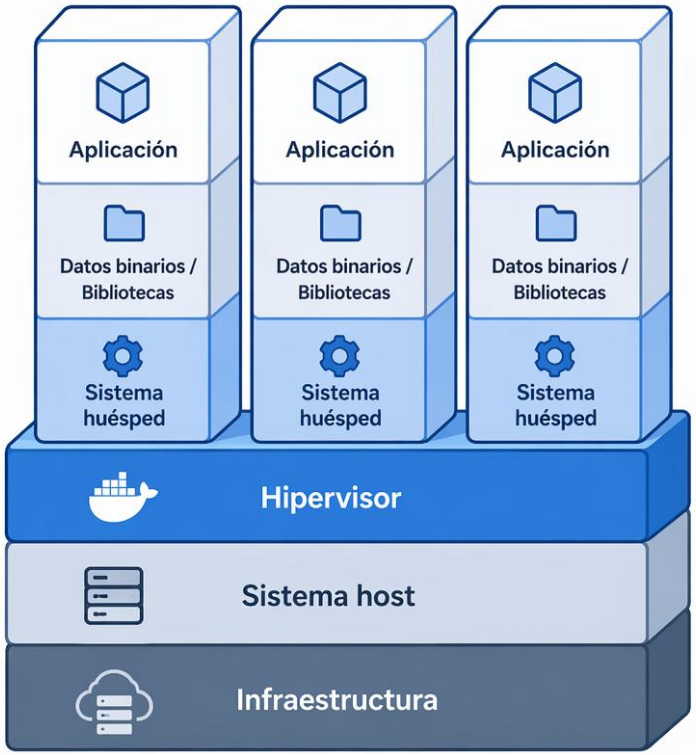
El concepto de contenedores no nace con Docker, existían tecnologías como LXC (Linux Containers), OpenVZ, Solaris Zones, que permiten la virtualización a nivel de sistema operativo LXC (Linux Containers)



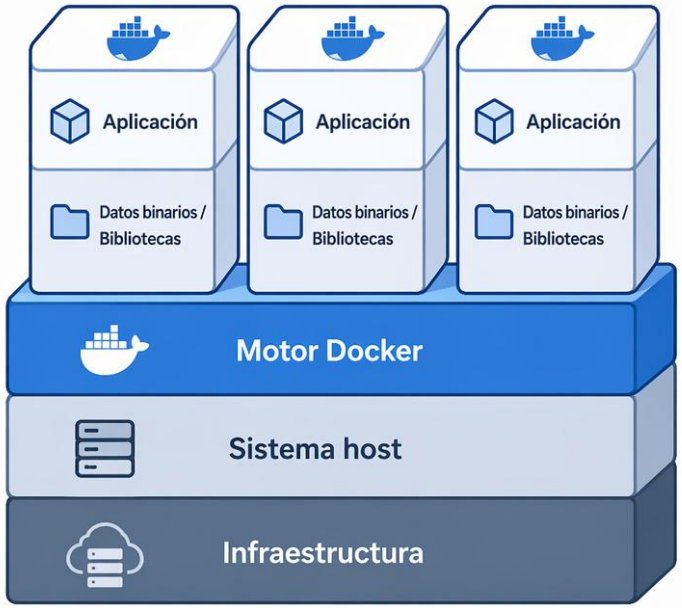
¿Qué es Docker?

Docker es una plataforma de código abierto que permite a los desarrolladores empaquetar aplicaciones junto con sus dependencias en contenedores ligeros, portátiles y autónomos.

Maquina Virtuales vs Contenedores



Máquina virtual



Contenedores Docker

Estandarización de los contenedores

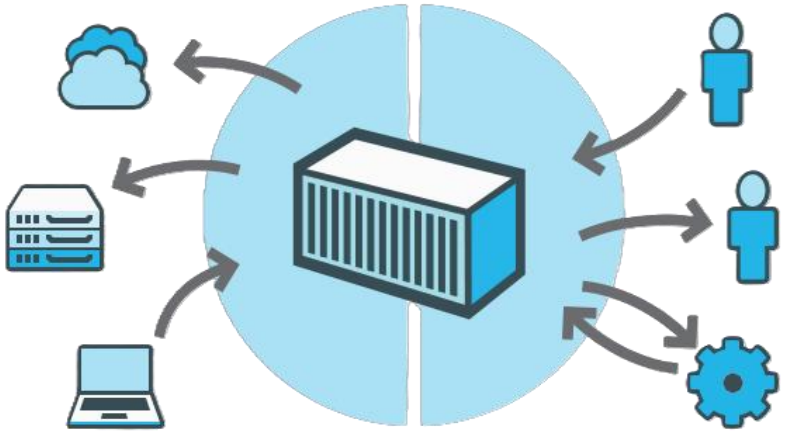
Uno de los estándares más importantes en el mundo de los contenedores es OCI (Open Container Initiative), que define una especificación común para los formatos de imagen y los entornos de ejecución de contenedores.

OCI fue creado en 2015 por un grupo de empresas líderes en tecnología, incluidas Docker, CoreOS, Google, Red Hat y VMware, con el objetivo de promover la interoperabilidad y la innovación en el ecosistema de contenedores.

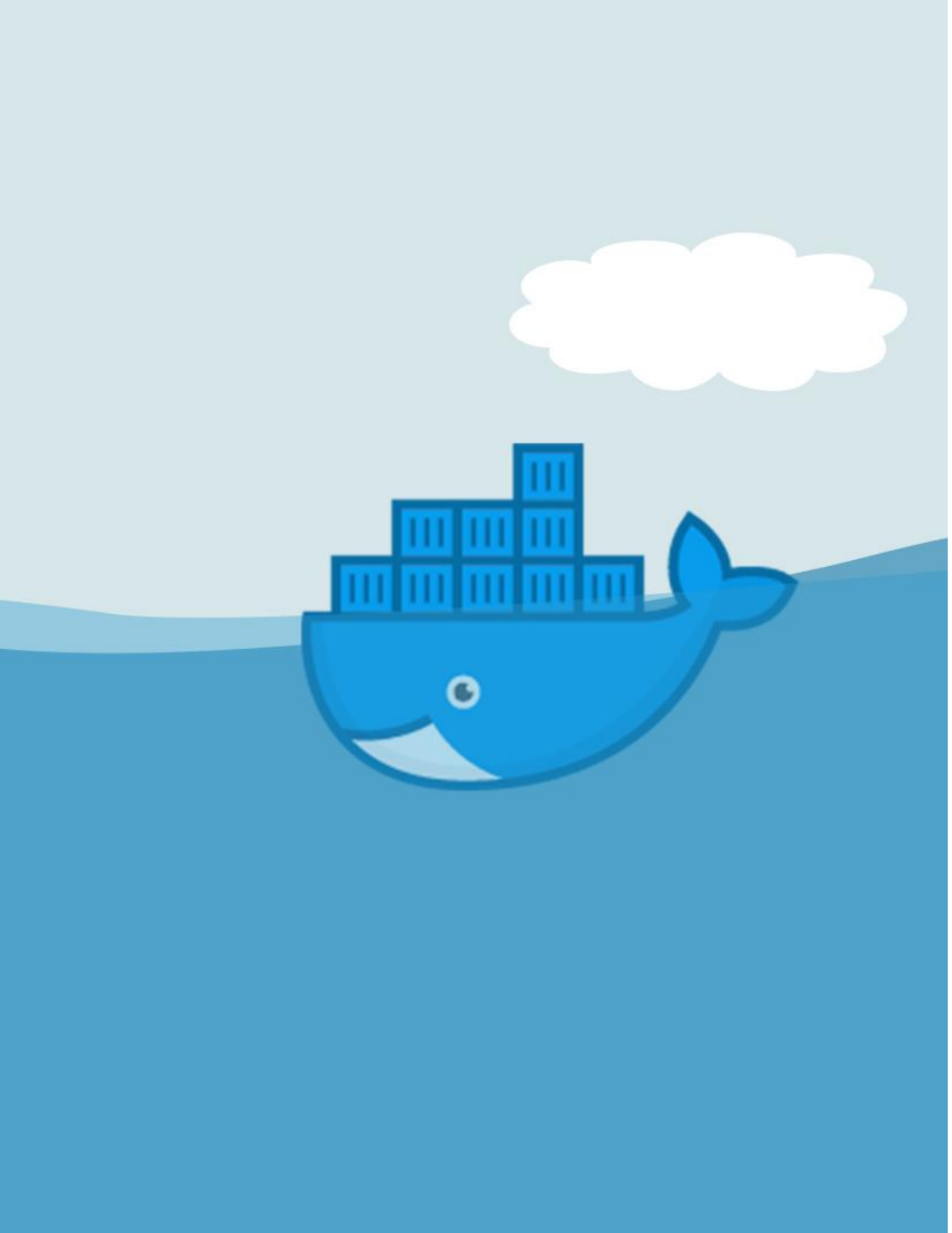
docker

¿Que problema resuelve Docker?

Casos de uso de los contenedores

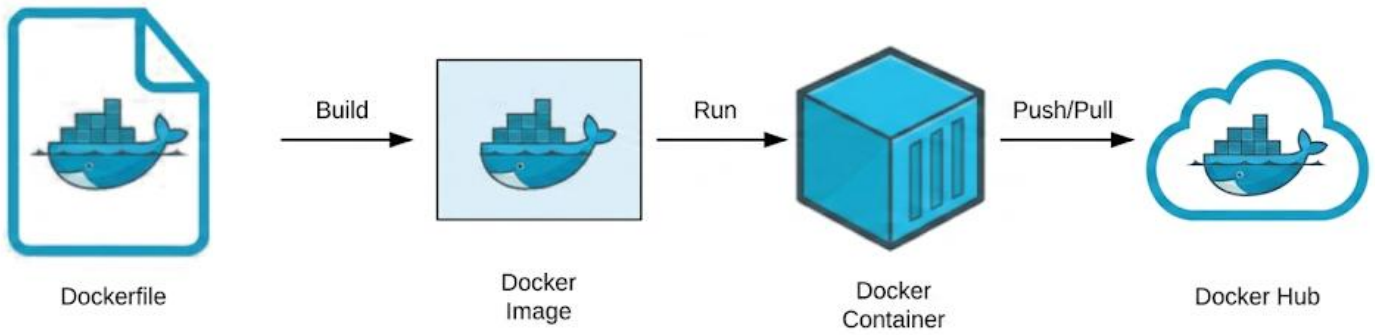


- Desarrollo y pruebas de aplicaciones
- Implementación de microservicios
- Implementación de aplicaciones en la nube
- IC/DC (Integración Continua/Despliegue Continuo)
- Aislamiento de aplicaciones
- Escalabilidad y alta disponibilidad
- Desarrollo multiplataforma



Conceptos básicos

Conceptos básicos

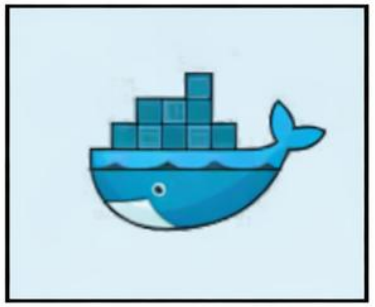


¿Qué es un contenedor en Docker?

Docker
Container

- Es una instancia de una imagen.
- Es un proceso que se ejecuta en un entorno aislado.

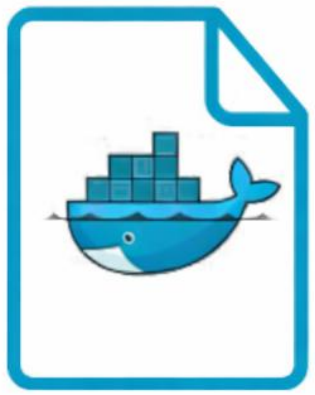
¿Qué es una imagen en Docker?



Docker
Image

- Es un archivo binario que contiene todos los elementos necesarios para ejecutar un contenedor.
- Es como una plantilla que se utiliza para crear contenedores.

¿Qué es un dockerfile en Docker?

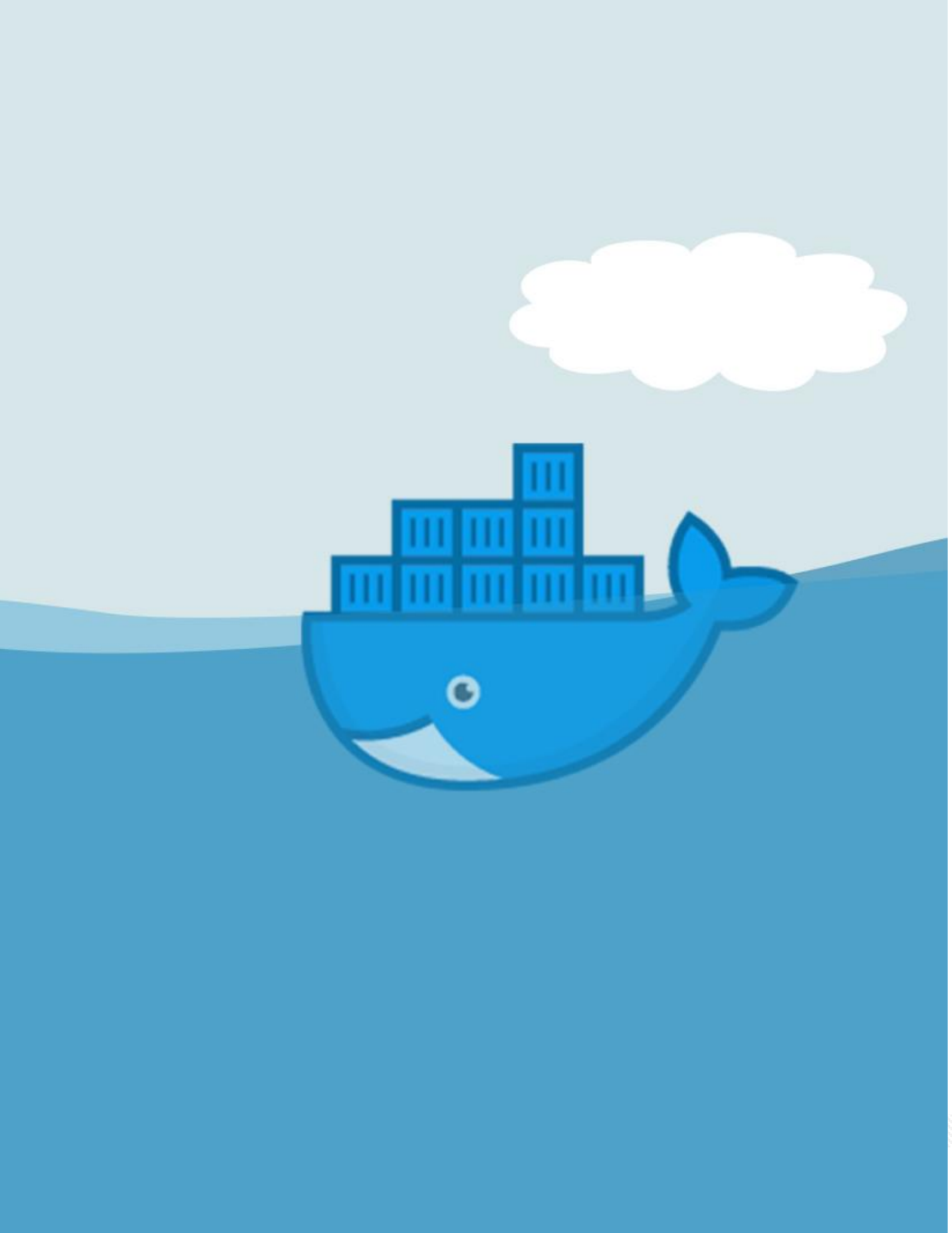


- Estos archivos, son un conjunto de instrucciones secuenciales que le especifican a Docker cómo construir una imagen.
- Permite usar múltiples instrucciones para instalar dependencias, copiar archivos, definir variables de entorno, etc.

¿Qué es dockerhub en Docker?

Docker Hub

- Es un repositorio de imágenes de contenedor. En Docker Hub, podemos encontrar imágenes de contenedores ya creadas por la comunidad, que podemos utilizar para nuestros proyectos.
- También podemos subir nuestras propias imágenes y compartirlas con la comunidad.



Ejecutar un contenedor

docker

Primer contenedor (hello-world)

```
docker run <imagen>
docker run hello-world
```

Arrancar un contenedor

```
docker run <imagen>
docker run nginx
docker ps
docker ps -a
```

El comando run

Junto al comando `docker run`, podemos utilizar una serie de opciones para personalizar el comportamiento del contenedor.

Algunas de las opciones más comunes son:

Parámetro	Descripción
`-d`	Arranca el contenedor en segundo plano (modo detached).
`-p`	Mapea un puerto del contenedor al puerto del host.
`-v`	Mapea un volumen del host al contenedor.
`--name`	Asigna un nombre al contenedor.
`--rm`	Elimina el contenedor automáticamente al detenerse.
`-e`	Define una variable de entorno.
`--env-file`	Cargas variables de entorno desde un archivo.

Proceso del contenedor y ejecución

```
docker run -d nginx
docker run -d --name mi-nginx nginx
docker logs <id_contenedor>
```

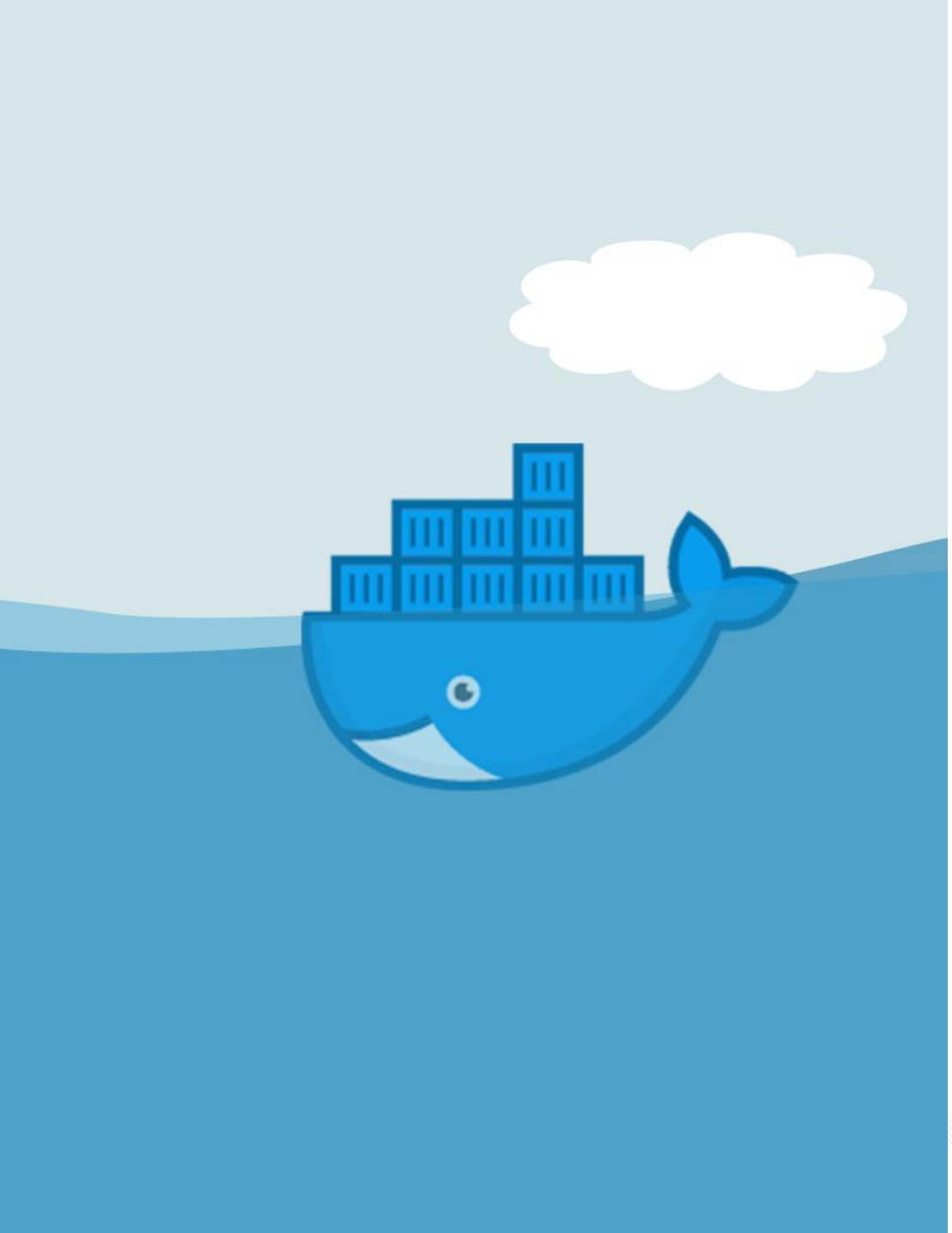
Política de reinicio

```
docker run --restart always nginx
```

Política	¿Qué hace realmente?	Cuándo usarla
`no`	No reinicia el contenedor	Pruebas, desarrollo
`always`	Siempre lo reinicia (aunque reinicies tu PC)	Producción
`unless-stopped`	Lo reinicia siempre, excepto si tú lo detuviste manualmente	Producción (más control)
`on-failure`	Solo reinicia si el contenedor falla (error)	Apps críticas
`on-failure:n`	Reinicia solo si falla, máximo n intentos	Evitar loops infinitos

Mapeo de puertos

```
docker run -d -p 8080:80 nginx
docker run -d -p 8080:80 -p 8081:81 nginx
docker run -d -p 8080-8085:80 nginx
```



Gestión de contenedores

docker

Conectarse a un contenedor

Por razones de depuración, a veces necesitamos conectarnos a un contenedor en ejecución y ejecutar un comando en él que no sea el principal. Para ello, podemos utilizar el comando `exec`.

```
docker exec <id_contenedor> ls
```

Conectarse a un contenedor

Lo más común, ejecutar el entorno de bash, sh o zsh en un contenedor y así obtener un terminal interactivo. Para poder permitir hay que utilizar los parámtros i, interactive y t, de terminal.

```
docker exec -it <id_contenedor> bash
```

Parar e Iniciar un contenedor

Parar.

```
docker ps

docker stop <id_contenedor>

docker stop <id_3_parcial>
```

Iniciar.

```
docker start <id_contenedor>

docker ps -a

docker ps -a | grep nginx
```

Reiniciar y Eliminar un contenedor

Reiniciar.

```
docker restart <id_cont…>
```

Eliminar.

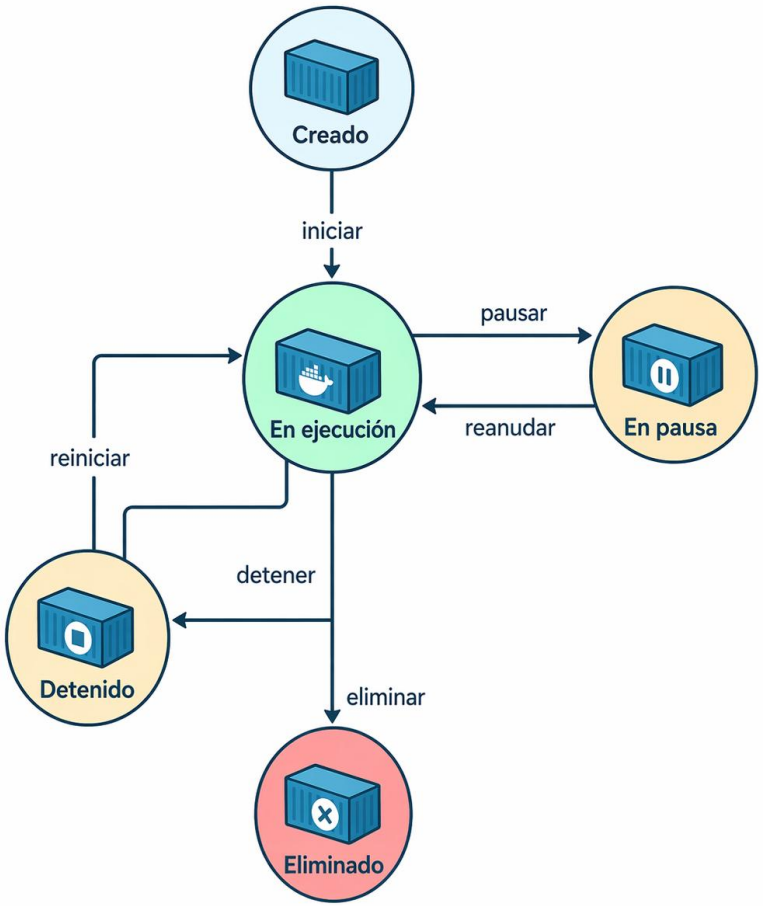
```
docker rm <id_cont…>
```

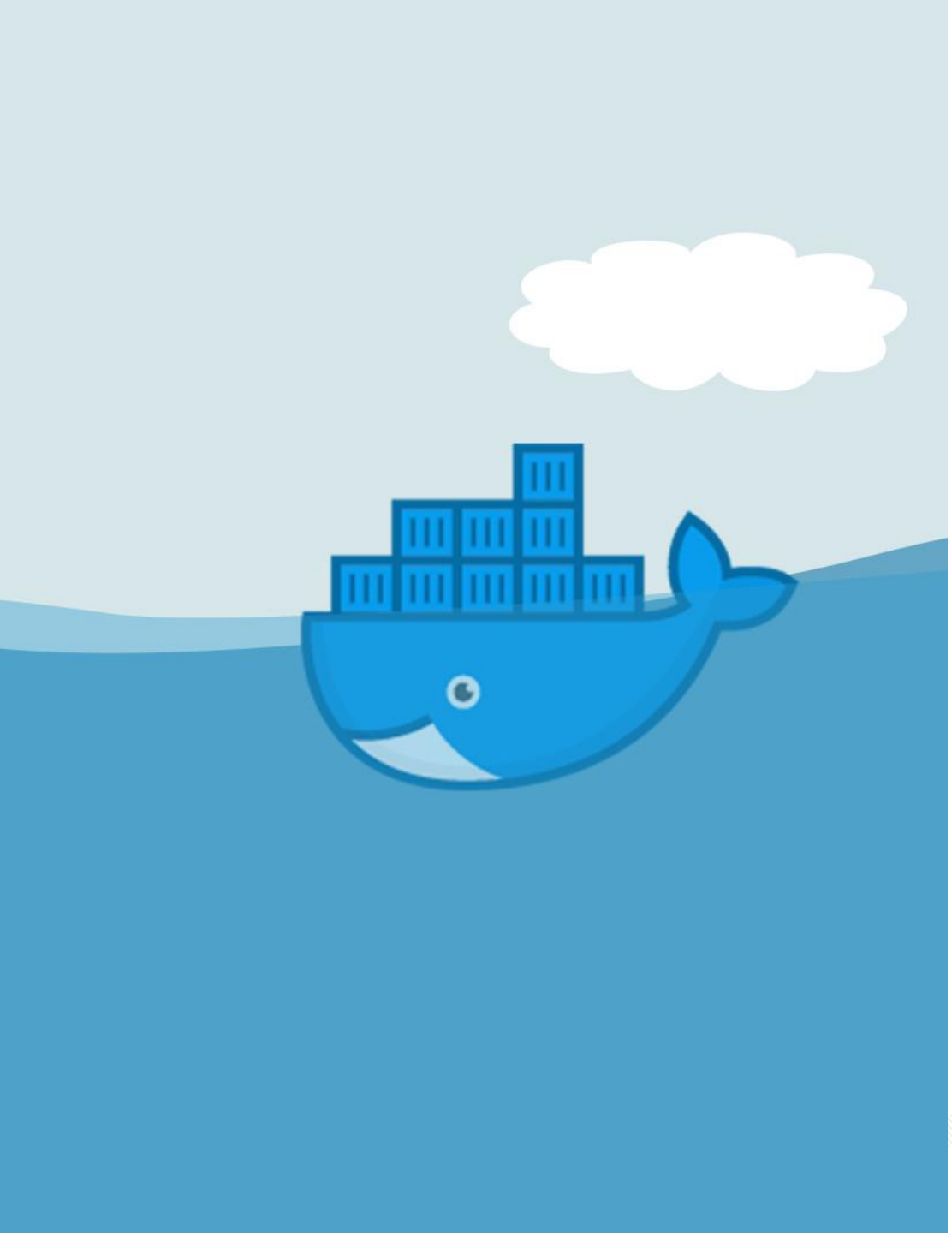
```
docker rm -f <id_cont…>
```

Prune de contenedores.

```
docker container prune
```

Ciclo de vida de un contenedor





Imágenes de contenedor

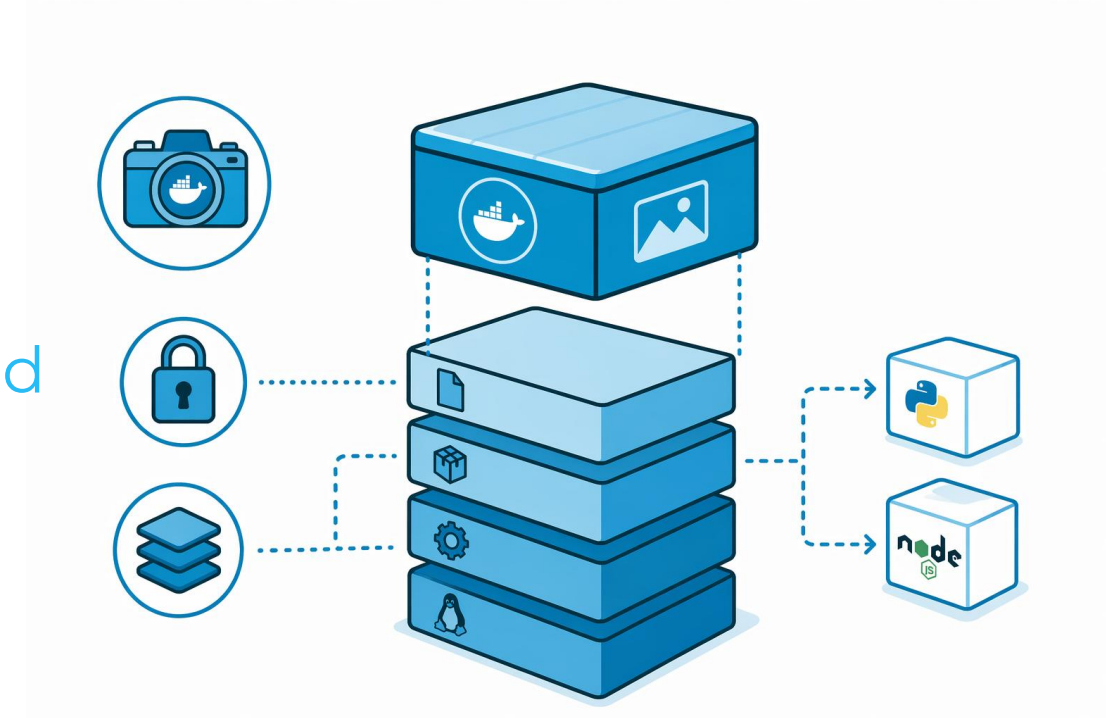
docker

Imagen

una imagen contiene todos los elementos necesarios para ejecutar un contenedor, funcionando como una plantilla que se utiliza para crear contenedores

Inmutabilidad

Capas



docker

Capas de una imagen

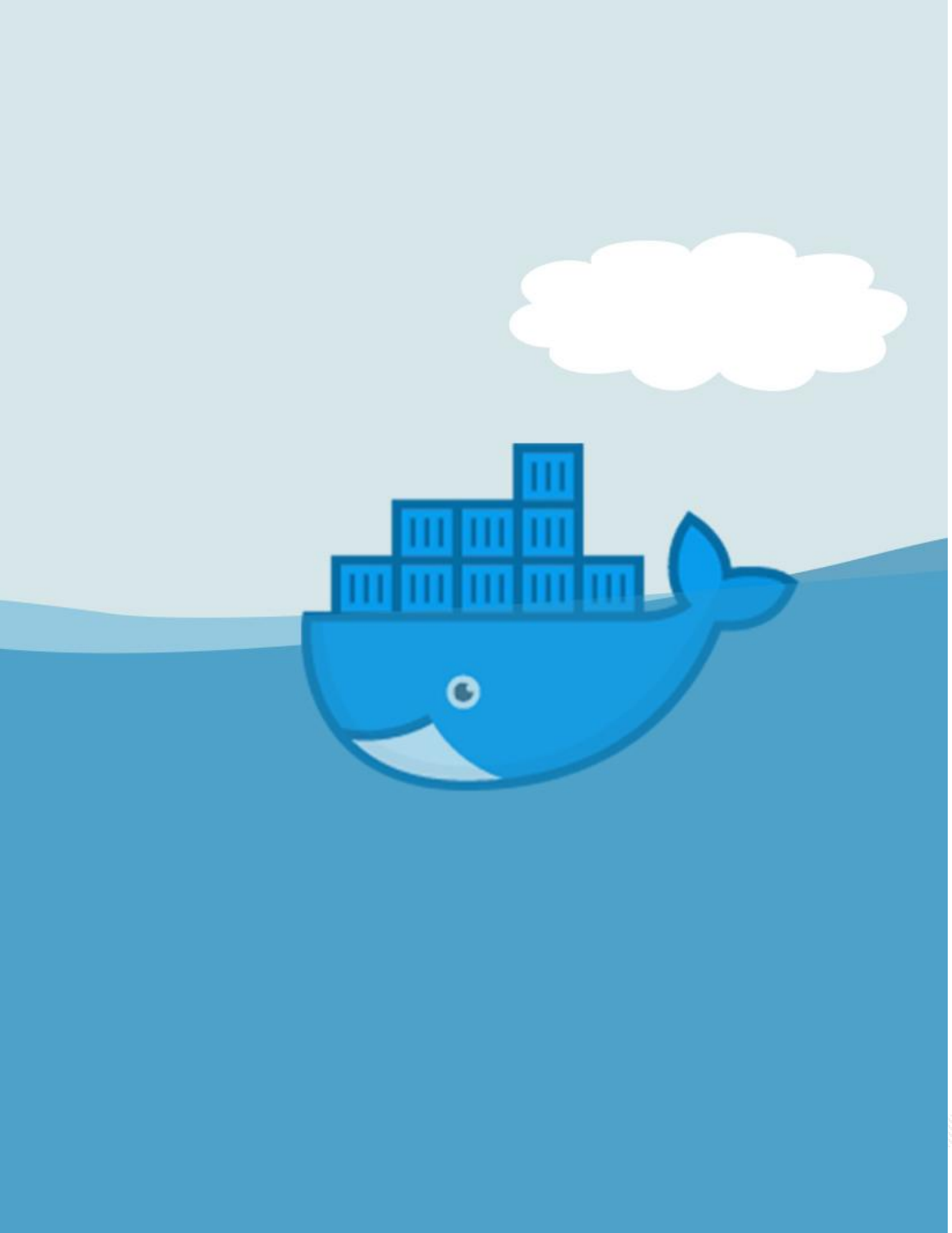
```
docker pull nginx
docker pull alpine

docker history nginx
docker history alpine

docker pull alpine:3.19
docker pull nginx:stable-alpine3.19
```

Borrar imagen

```
docker images
docker rmi <name>
docker rmi <id_imagen>
```

Prune de imágenes.

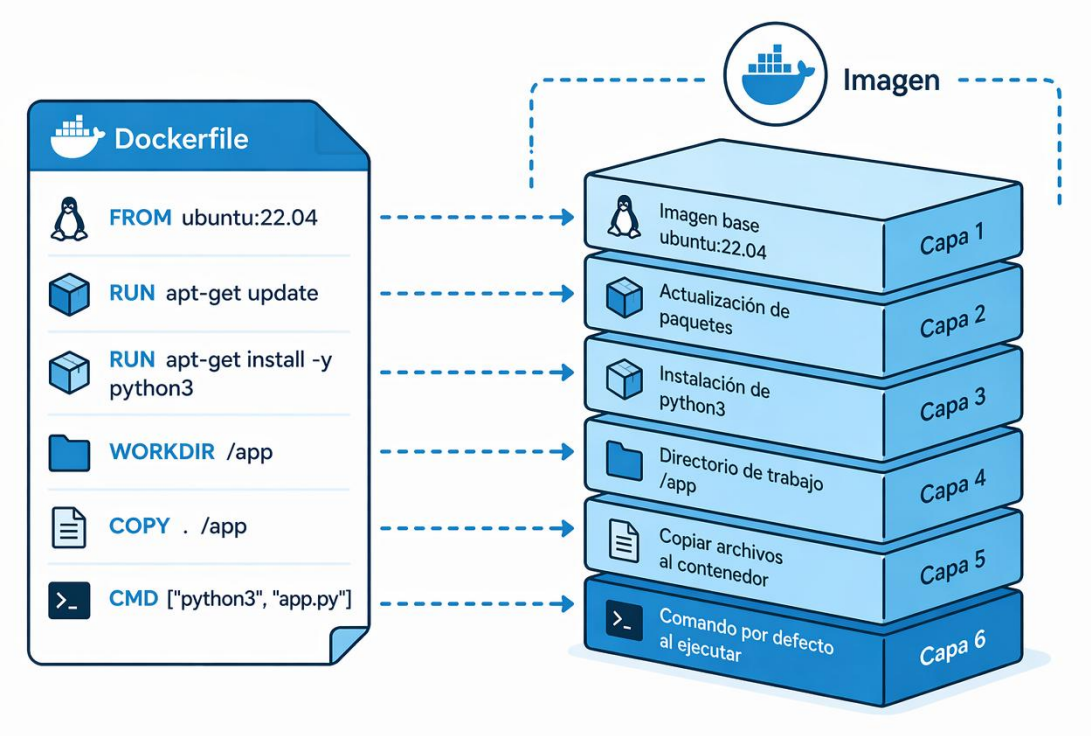
```
docker image prune
```



Dockerfiles y build

docker

Dockerfile



Un Dockerfile es un archivo de texto que contiene una serie de instrucciones que Docker leerá y ejecutará para construir una imagen de contenedor.

Cada instrucción en un Dockerfile crea una capa en la imagen.

Instrucciones mas comunes

FROM
Indica la imagen base que se utilizará para construir la nueva imagen.

RUN
Ejecuta comandos en la imagen.

COPY
Copia archivos o directorios desde el host a la imagen.

WORKDIR
Establece el directorio de trabajo por defecto para el resto de instrucciones.

CMD
Define el comando que se ejecutará cuando se inicie un contenedor a partir de la imagen.

ENTRYPOINT
Define el comando que se ejecutará cuando se inicie un contenedor a partir de la imagen, pero no se puede sobreescribir.

docker

Docker Build

```
docker build -t <nombre_imagen> <directorio_contexto>
```

Asigna un nombre (y opcionalmente etiqueta) a la imagen creada.

Es el directorio que se envía al docker daemon como contexto para construir la imagen.

1. **Directorio de contexto**
   Contiene el Dockerfile y los archivos necesarios.
2. **Ejecuta el comando**
   `docker build -t mi-app .`
3. **Docker envía el contexto**
   El contenido del directorio se envía al Docker daemon.
4. **Se construye la imagen**
   Se ejecutan las instrucciones del Dockerfile paso a paso.
5. **Imagen creada**
   Se crea la imagen con el nombre especificado.

https://github.com/ernaestrujillo/TallerDocker

Ejemplo básico ngnix

```
# Utilizamos la imagen oficial de Nginx
FROM nginx:alpine

# Copiamos el fichero index.html al directorio /usr/share/nginx/html
COPY index.html /usr/share/nginx/html/index.html
```

```
1. docker build -t mi-nginx .
2. docker image
3. docker run -d --name mi-nginx -p 8080:80 mi-nginx
```

https://github.com/ernaestrujillo/docker/index.html

docker

Ejemplo básico python

```
# Utilizamos la imagen oficial de Python
FROM python:alpine
# Establecemos el directorio de trabajo
WORKDIR /app
# Desactiva el buffer de Python para ver logs en tiempo real
ENV PYTHONUNBUFFERED=1
# Copiamos el script de Python al directorio de trabajo
COPY script.py .
# Ejecutamos el script de Python
CMD ["python", "script.py"]
```

```
1. docker build -t mi-python .
2. docker image
3. docker run -d --name mi-python mi-python
4. docker logs -f mi-python
```

Docker NodeJS

```
# Utilizamos la imagen oficial de Node.js en su versión alpine (ligera)
FROM node:alpine
# Establecemos el directorio de trabajo dentro del contenedor
WORKDIR /app
# Copiamos el script al directorio de trabajo
COPY script.js .
# Exponemos el puerto 3000 para acceder a la API
EXPOSE 3000
# Ejecutamos el script con Node.js
CMD ["node", "script.js"]
```

Docker NodeJS con Hono

Hono

```
# Utilizamos la imagen oficial de Node.js en su versión alpine (ligera)
FROM node:alpine
# Establecemos el directorio de trabajo dentro del contenedor
WORKDIR /app
# Copiamos el package.json e instalamos dependencias
COPY package.json .
RUN npm install
# Copiamos el script al directorio de trabajo
COPY script.js .
# Exponemos el puerto 3000 para acceder a la API
EXPOSE 3000
# Ejecutamos el script con Node.js
CMD ["node", "script.js"]
```


docker

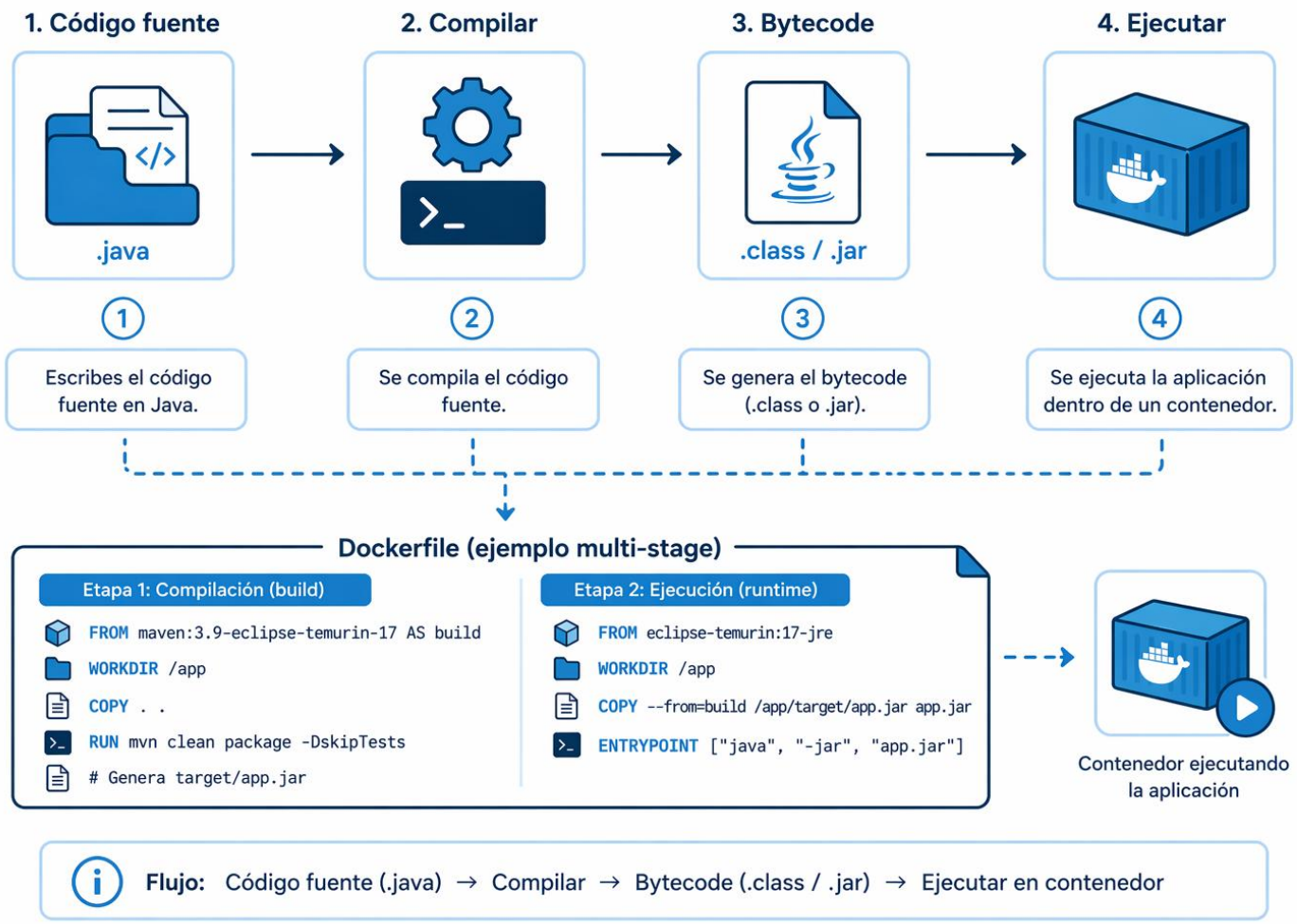
Ejemplo python

FastAPI

```
# Utilizamos la imagen oficial de Python en su versión alpine (ligera)
FROM python:alpine
# Establecemos el directorio de trabajo dentro del contenedor
WORKDIR /app
# Copiamos e instalamos las dependencias
COPY requirements.txt .
RUN pip install -r requirements.txt --no-cache-dir
# Copiamos el código al directorio de trabajo
COPY main.py .
# Exponemos el puerto 8000
EXPOSE 8000
# Ejecutamos la aplicación con uvicorn
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```


docker

Lenguajes compilados

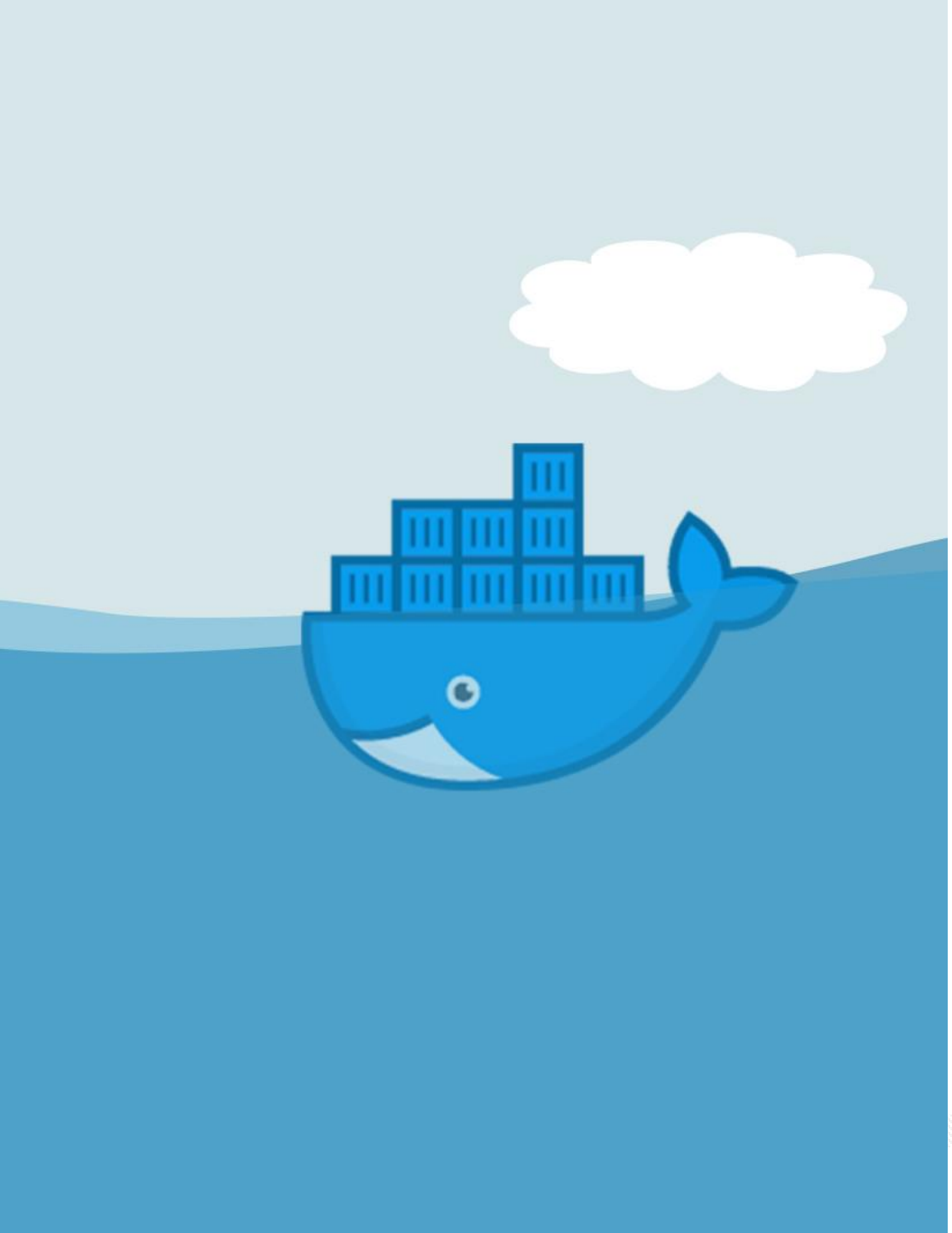


Ejemplo java

```
# ---- Etapa 1: compilar ----
FROM maven:3.9-eclipse-temurin-21-alpine AS builder

WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline

COPY Main.java src/main/java/Main.java
RUN mvn package -q
# ---- Etapa 2: ejecutar ----
FROM eclipse-temurin:21-jre-alpine

WORKDIR /app
COPY --from=builder /app/target/hola-docker-1.0.jar app.jar

EXPOSE 3000
CMD ["java", "-jar", "app.jar"]
```

Ejemplo aspnet core

```
# ---- Etapa 1: compilar ----
FROM maven:3.9-eclipse-temurin-21-alpine AS builder

WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline

COPY Main.java src/main/java/Main.java
RUN mvn package -q

# ---- Etapa 2: ejecutar ----
FROM eclipse-temurin:21-jre-alpine

WORKDIR /app
COPY --from=builder /app/target/hola-docker-1.0.jar app.jar

EXPOSE 3000
CMD ["java", "-jar", "app.jar"]
```

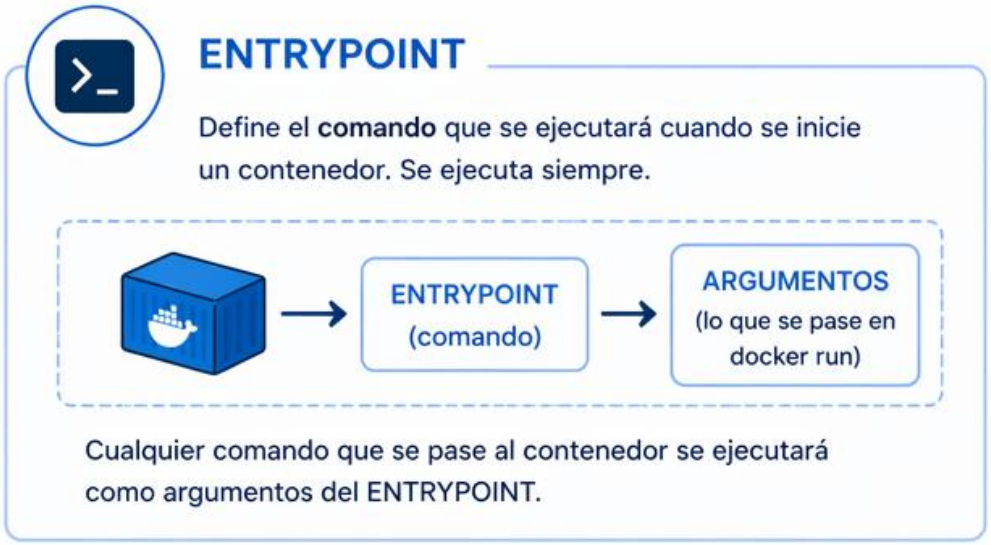


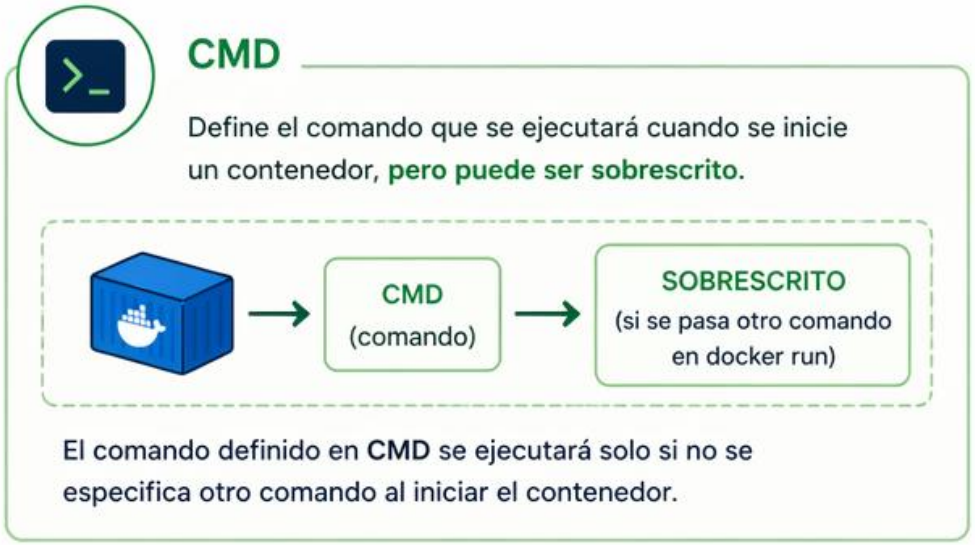
Entrypoints, argumentos y variables de entorno

docker

ENTRYPOINT y CMD

En Dockerfile, las instrucciones ENTRYPOINT y CMD son las que definen qué comando se ejecutará cuando se inicie un contenedor. Tienen una funcionalidad similar, pero con una diferencia importante.





Ejemplo ENTRYPOINT y CMD

```
FROM alpine
ENTRYPOINT ["echo", "Hola"]
CMD ["Mundo"]
```

```
1. docker build -t saludador .
2. docker run saludador
```

```
1. docker run saludador "desde Docker"
2. docker run saludador
```

```
Eliminar los contenedores
```


docker

Argumentos y Variables de Entorno

Además de ENTRYPOINT y CMD, podemos pasar argumentos a nuestro proceso de construcción y definir variables de entorno en nuestro Dockerfile.

ARGUMENTOS (ARG)

1. **Se declaran con la instrucción ARG**
   `ARG NOMBRE=valor_por_defecto`
2. **Se usan durante la construcción**
   Sus valores pueden ser usados en instrucciones como RUN, COPY, ADD, etc.
3. **Se pasan con `--build-arg`**
   Se especifican al ejecutar `docker build` para sobrescribir el valor por defecto.
4. **Varían la imagen generada**
   Permiten adaptar el proceso de construcción a diferentes entornos sin modificar el Dockerfile.

VARIABLES DE ENTORNO (ENV)

1. **Se declaran con la instrucción ENV**
   `ENV NOMBRE=valor`
2. **Disponibles en tiempo de ejecución**
   Las variables están presentes cuando el contenedor se está ejecutando.
3. **Se pueden sobrescribir al ejecutar**
   Usando la opción `-e` o `--env` en `docker run`.
4. **Útiles para configuración**
   Ideales para configuración de aplicaciones, rutas, claves, modos, etc.

Importante:

- ARG solo existe en tiempo de construcción.
- ENV existe en tiempo de ejecución.
- Usa ARG para parametrizar la construcción de la imagen.
- Usa ENV para configurar el comportamiento del contenedor.

Ejemplos Argumentos

```
FROM alpine

# Declarar un argumento con un valor predeterminado
ARG NOMBRE="Mundo"

# Crear un archivo que contenga el mensaje personalizado
RUN echo "Hola $NOMBRE" > /message

# Comando predeterminado para mostrar el contenido del archivo
CMD ["cat", "/message"]
```

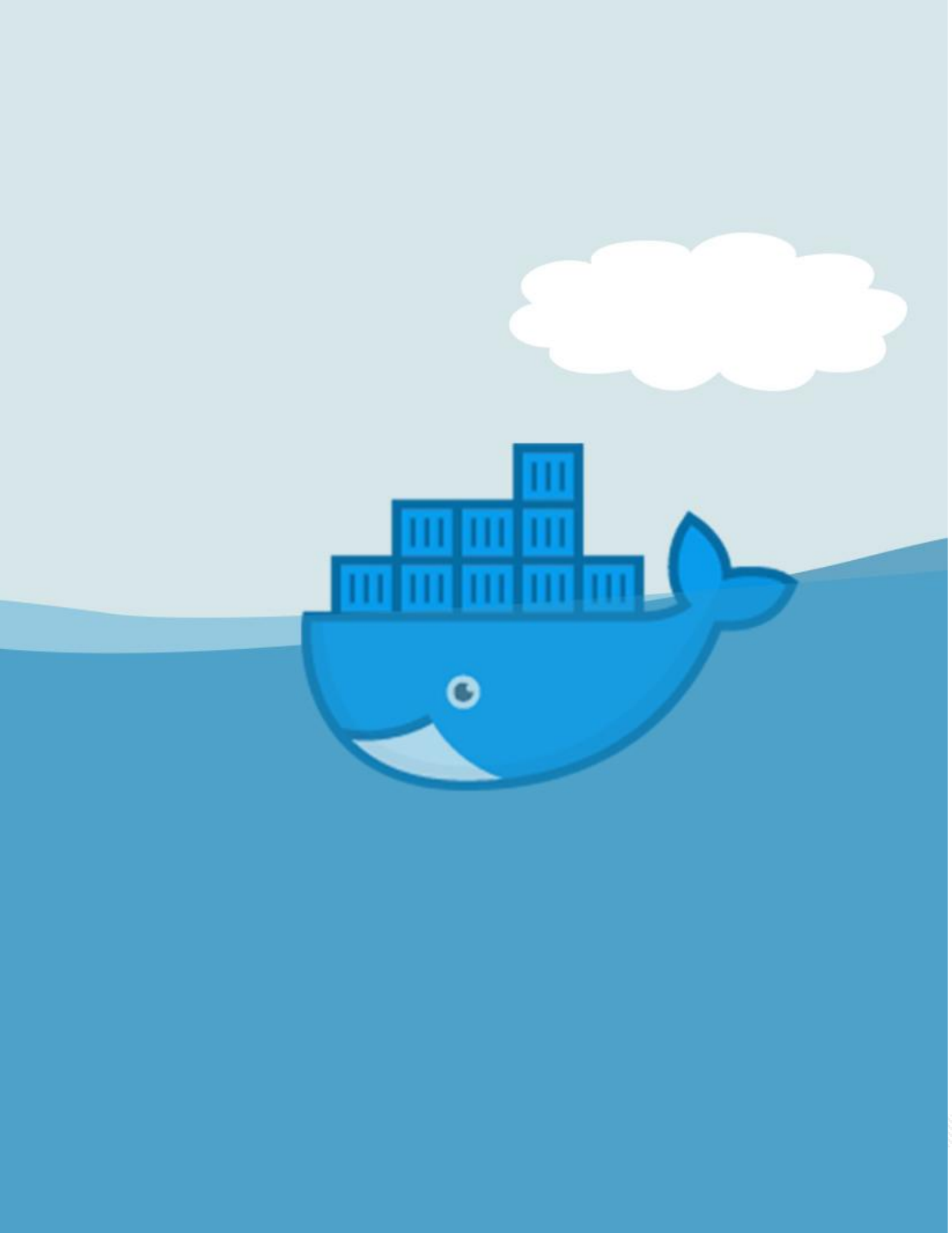
```
1. docker build -t saludador .
2. docker run saludador
3. docker build --build-arg NOMBRE="Desde Docker" -t saludador .
4. docker run saludador
```

Ejemplos Variables de entorno

```
FROM node:alpine
# Establecemos el directorio de trabajo
WORKDIR /app
# Copiamos el script
COPY script.js .
# Ejecutamos el script
CMD ["node", "script.js"]
```

```
if (process.env.DEBUG) {
    console.log("Modo de depuración activado");
} else {
    console.log("Modo de depuración desactivado");
}
```

```
1. docker build -t hola-node-env .
2. docker run hola-node-env
3. docker run -e DEBUG=true hola-node-env
```

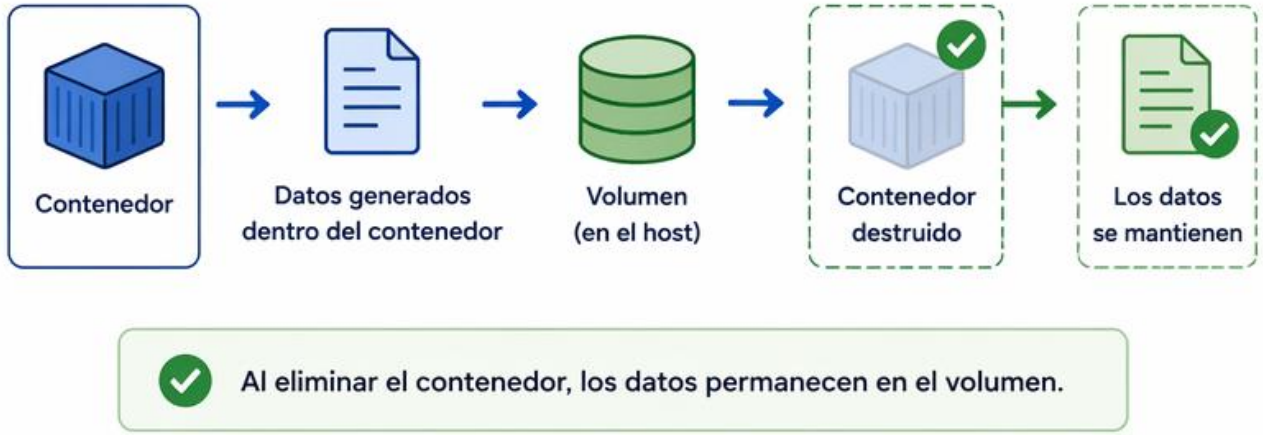


Volúmenes, persistencia y montaje

docker

Gestión de volúmenes

Los contenedores están diseñados para ser efímeros, es decir, que se puedan crear y destruir fácilmente. Sin embargo, en muchas ocasiones necesitaremos que los datos que se generen en un contenedor sean persistentes. Para ello, Docker nos proporciona los volúmenes.



Gestión de volúmenes

```
#crear volumen
docker volume create mi-volume
#listar volumenes
docker volume ls
#inspecionar el volumen
docker volume inspect mi-volume
#eliminar un volumen
docker volume rm mi-volumen
```

Ejemplo 1 volúmenes

1. `docker volume create mi-volume`
2. `docker run -d --name mi-contenedor -v mi-volumen:/datos ubuntu bash`
3. `ls`
4. `touch txt`
5. `ls -la`

Ejemplo 2 volúmenes

```
# Crear el volumendocker
docker volume create ng-volumen

# Correr nginx con el volume
docker run -d --name ng-contenedor -p 8080:80 -v ng-
volumen:/usr/share/nginx/html nginx

docker cp index.html ng-contenedor:/usr/share/nginx/html/
docker cp about.html ng-contenedor:/usr/share/nginx/html/
docker cp contact.html ng-contenedor:/usr/share/nginx/html/
```

Montaje de archivos y directorios

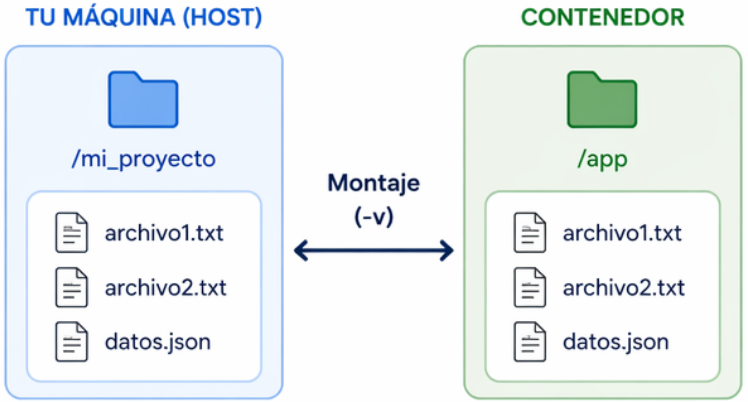
Montar un directorio permite compartir archivos entre tu máquina local y el contenedor.
Los cambios se reflejan en ambos lados.

1 COMANDO

```
docker run -v <directorio_local>:<directorio_contenedor> <imagen>
```

- <directorio_local>: ruta absoluta en tu máquina.
- <directorio_contenedor>: ruta dentro del contenedor.
- Los cambios en uno se reflejan en el otro.

2 ¿CÓMO FUNCIONA?



3 EJEMPLO

```
docker run -it -v /home/usuario/mi_proyecto:/app node:18
```

- El directorio `/home/usuario/mi_proyecto` de tu máquina se monta en `/app` dentro del contenedor.
- Cualquier cambio en los archivos se verá en ambos lados.

USOS COMUNES

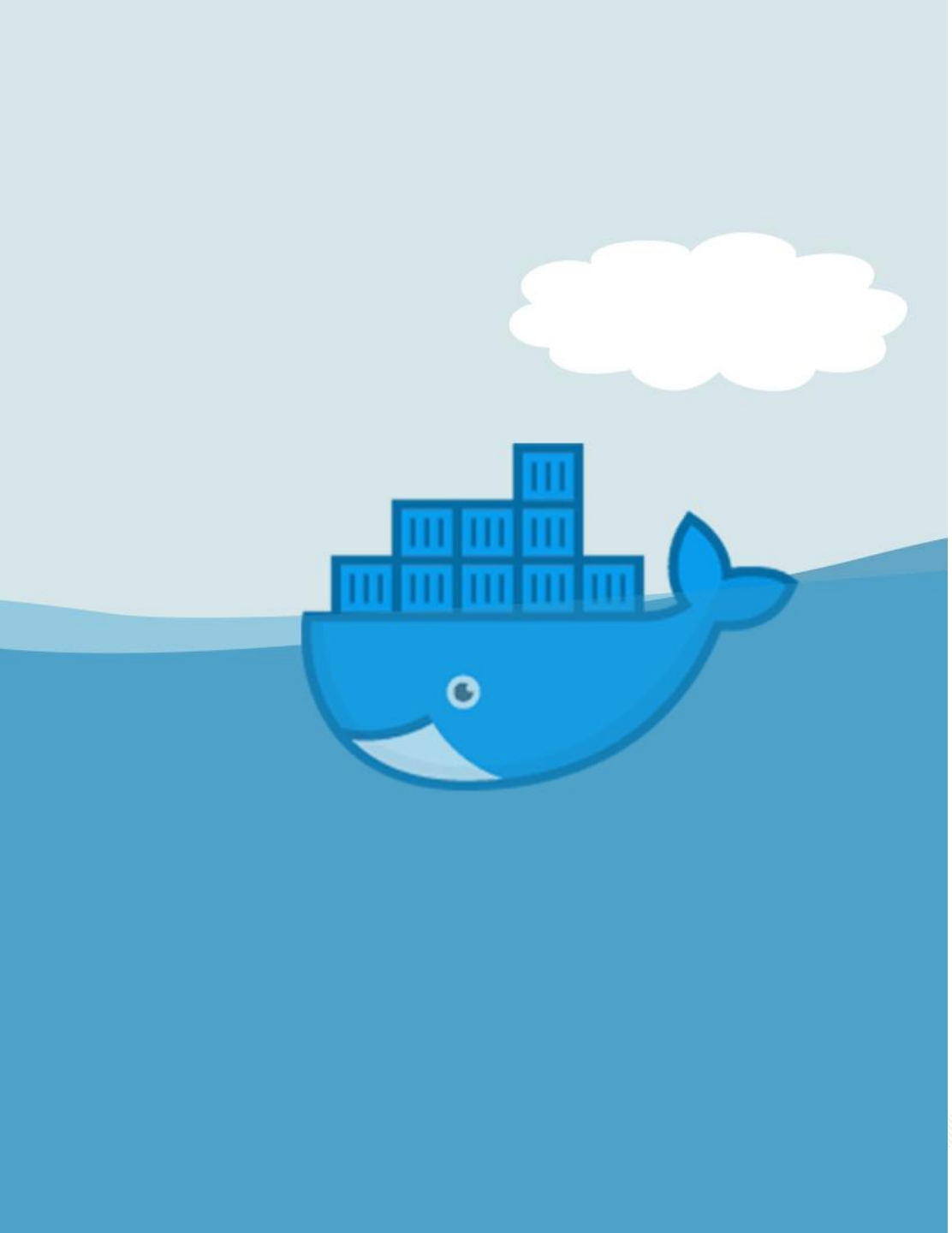
- ✓ Desarrollo: sincronizar código fuente.
- ✓ Configurar aplicaciones.
- ✓ Persistir datos generados por la aplicación.

Ejemplo montaje

- `docker build -t lector-volumen .`

- `docker run -d --name lector -p 3000:3000 -v C:\Users\vizue\datos:/datos lector-volume`

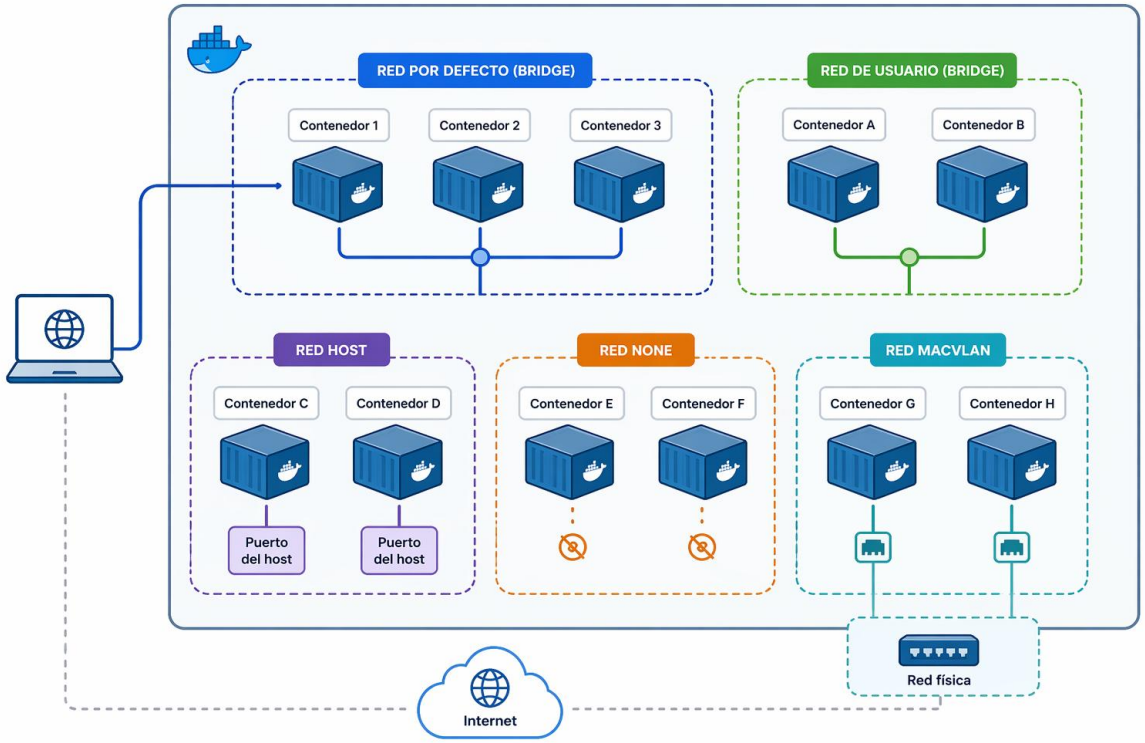
```
# Ver todos los archivos del volumen
http://localhost:3000/archivos

# Leer un archivo específico
http://localhost:3000/archivos/saludo.txt
```



Redes Docker

Redes Docker

Las redes en Docker son un tema muy importante, ya que nos permiten conectar contenedores entre sí o aislarlos unos de otros

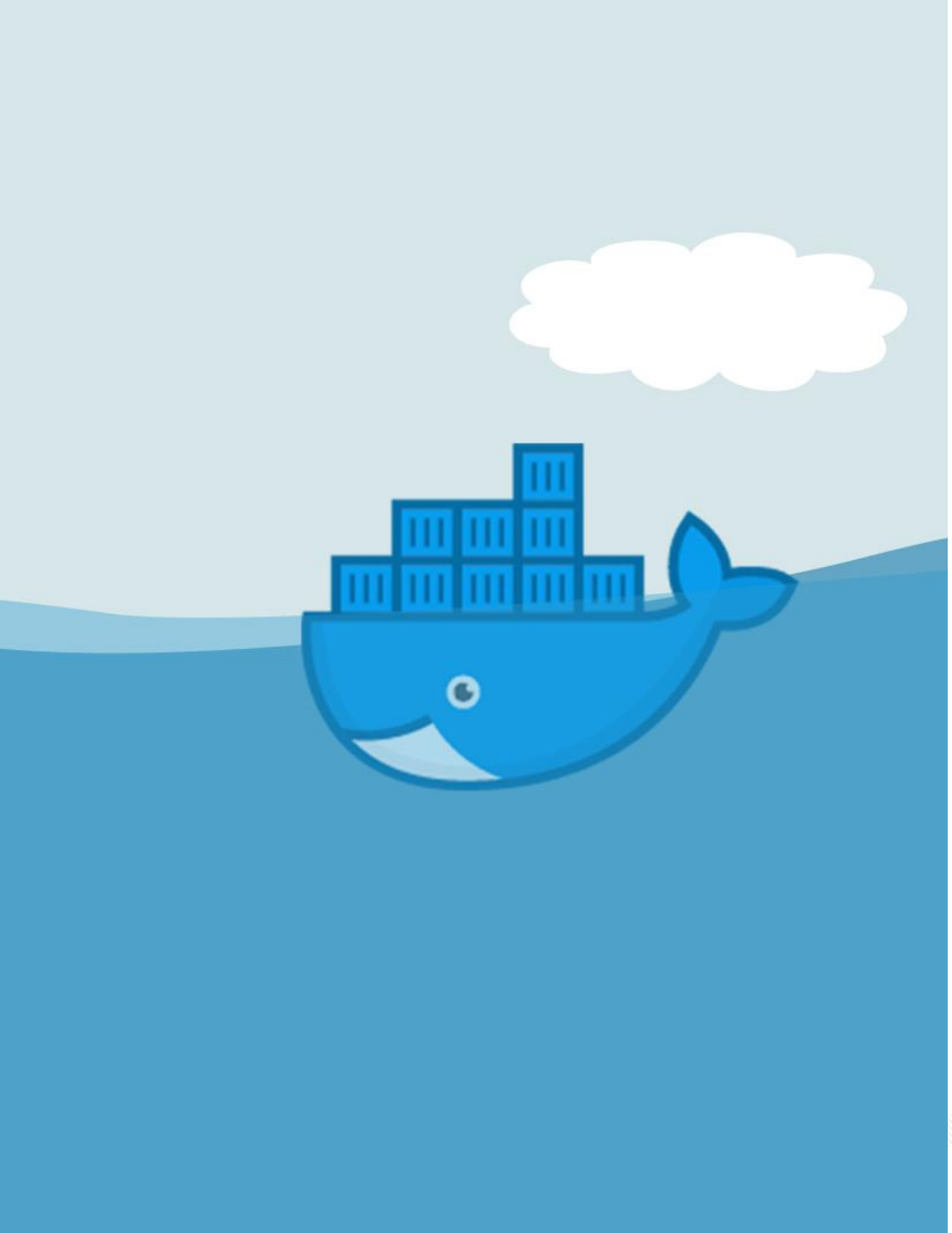


Redes Docker

Red	Descripción	Caso de uso
`bridge`	Red por defecto, comunicación entre contenedores en el mismo host	Desarrollo local, múltiples contenedores en una misma máquina
`host`	Los contenedores comparten la red del host directamente	Cuando necesitas máximo rendimiento de red, con precaución por seguridad
`overlay`	Comunicación entre contenedores en diferentes hosts	Ambientes distribuidos, Docker Swarm
`macvlan`	Asigna una dirección MAC al contenedor, se comporta como dispositivo físico	Cuando el contenedor necesita aparecer como un dispositivo real en la red
`none`	Sin red	Contenedores aislados que no necesitan comunicación de red

Gestión de redes

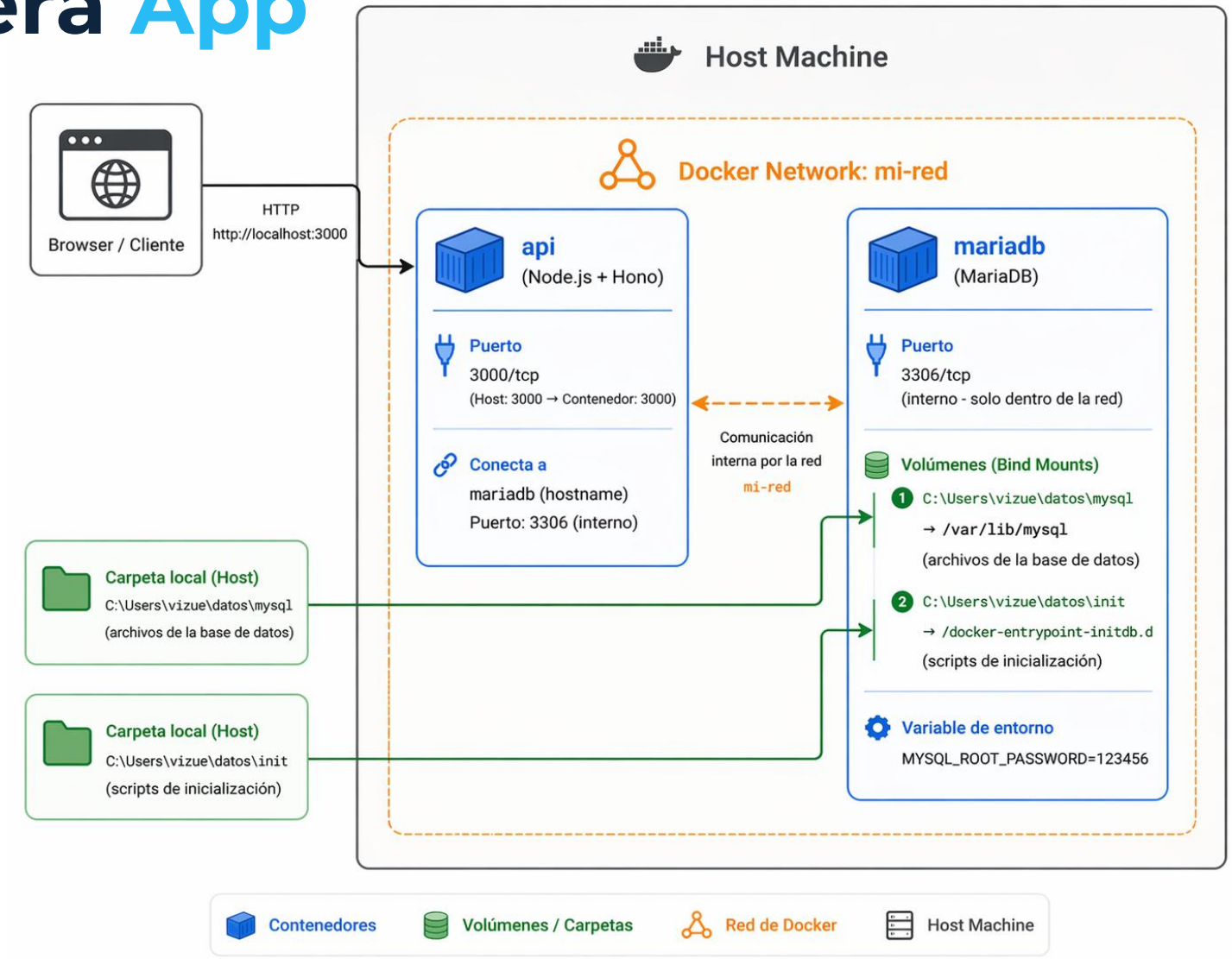
```
#crear una red
docker network create mi-red
#listar redes
docker network ls
#inspecionar el volumen
docker network inspect mi-red
#eliminar un volumen
docker network rm mi-red
#conectar una red a un contenedor
docker run -d --name mi-contenedor --network mi-red nginx
docker network connect mi-red mi-contenedor
```



Tu primera App

docker

Tu primera App



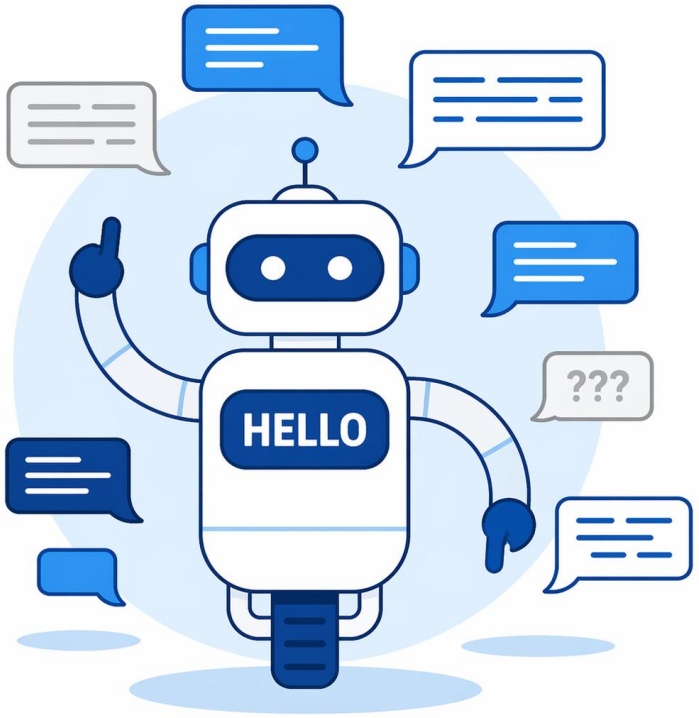
Tu primera App

```
docker network create mi-red-api

docker run -d --name db_maria --network mi-red-api -e
MYSQL_ROOT_PASSWORD=123456  -v
C:\Users\vizue\datos\mysql:/var/lib/mysql -v
C:\Users\vizue\datos\init:/docker-entrypoint-initdb.d
mariadb:latest

docker build -t api-contactos ./api

docker run -d --name api-contactos --network mi-red-api -p
3001:3000 api-contactos
```

Otras formas de ejecutar el contenedor

```
docker run -d --name api --network mi-red-api -p 3001:3000 -e
DB_HOST=db -e DB_USER=root -e DB_PASSWORD=123456 -e
DB_NAME=agenda -e DB_PORT=3306 api-contactos

docker run -d --name api --network mi-red-api -p 3001:3000 --
env-file .env api-contactos
```

¿Preguntas?



¡Gracias!